

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – ERROR ANALYSIS

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO2 – Manipulation	Assessment:	Assessment Tool 3 – ERROR ANALYSIS
Weightage:	30% of CIA		
CO5 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	T. Bhanu Pavan Varma	Reg. No.:	192425380
Section / Batch:	B. Tech AI&ML (2024-2028)	Date:	

Code of Conduct

I T. Bhanu Pavan Varma(192425380) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- Read all questions carefully before attempting the answers.
- Answer any 4 questions (2 Scenario-Based & 2 Programming-Based). Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Inflectional Morphology and Derivational Morphology	Error analysis of morphological decomposition in unhappiness, happiness, and happier; identification of roots, prefixes, suffixes, inflectional and derivational morphemes, and correction of incorrect morphological classifications.	Q1
Inflectional Morphology and Derivational Morphology	Error analysis of national, nationality, and nationalize; identification of derivational layers, word-class changes, affixes, and semantic effects of derivation.	Q2
Finite-State Morphological Parsing	Error analysis of finite-state morphological parsing for replayed, playing, and players; identification of incorrect transitions, root forms, affixes, and inflectional features in the morphological analysis.	Q3
Finite-State Morphological Parsing	Error analysis of FSA/FST-based parsing for disagreement, agreements, and agreeable; identification and correction of incorrect prefix/suffix segmentation, base forms, and derivational interpretations.	Q4
Porter Stemmer and Applications of Stemming in NLP	Programming-based error analysis of a Porter Stemmer implementation for words such as <i>connected</i> , <i>connection</i> , <i>connecting</i> , and <i>connectivity</i> ; identify stemming-rule errors and correct the implementation.	Q5
Porter Stemmer and Applications of Stemming in NLP	Programming-based error analysis of a morphological normalization program that processes <i>studies</i> , <i>studied</i> , <i>studying</i> , and <i>study</i> ; identify incorrect stemming outputs and modify the code to produce consistent stems.	Q6

Porter Stemmer and Applications of Stemming in NLP	Programming-based error analysis of a document indexing/stemming program; identify errors in applying Porter stemming to a collection of words, correct the code, and explain how the correction improves search and information retrieval.	Q7
---	--	-----------

Annexure A – Error Analysis

Scenario-Based Error Analysis (1 – 4)

Question 1 – Morphological Analysis Error in Biomedical Dataset

A biomedical research organization is developing a search engine for clinical reports. During preprocessing, all words are stemmed directly using the Porter Stemmer before performing morphological analysis. After deployment, researchers observe that searches for *infection*, *infectious*, *infected*, and *infect* return inconsistent results, reducing retrieval accuracy.

Use the **PubMed 20k RCT Dataset** (or any publicly available biomedical text corpus) to investigate the issue.

Tasks

1. Perform preprocessing on the dataset.
2. Apply Porter Stemmer and record the generated stems.
3. Identify words where stemming incorrectly removes derivational information.
4. Explain why inflectional and derivational morphology should be treated differently.

Question 1 – Morphological Analysis Error in Biomedical Dataset

Aim

To show how Porter Stemmer can remove useful morphological information from biomedical words.

1. Preprocessing + Porter Stemming

```
from nltk.stem import PorterStemmer
import re
```

```
ps = PorterStemmer()
```

```
text = """
Infection is a common medical condition.
Infectious diseases can spread quickly.
The patient was infected with bacteria.
Researchers study how infections develop.
"""
```

```
words = re.findall(r'\b[a-zA-Z]+\b', text.lower())
```

```
print("Original Word -> Stem")
```

```
for word in words:
    print(word, "->", ps.stem(word))
```

Important output

For the important words:

```
infection -> infect
```

infectious -> infecti
infected -> infect
infect -> infect
infections -> infect

The problem is:

infection -> infect
infectious -> infecti
infected -> infect
infect -> infect

infectious -> infecti is particularly problematic because **"infecti" is not a meaningful English word.**

2. Find problematic biomedical words

```
from nltk.stem import PorterStemmer
```

```
ps = PorterStemmer()
```

```
words = [  
    "infect",  
    "infected",  
    "infection",  
    "infections",  
    "infectious",  
    "infectivity",  
    "infectiousness"  
]
```

```
for word in words:
```

```
    print(f'{word:20} -> {ps.stem(word)}')
```

3. Inflectional vs Derivational Morphology

Inflectional morphology changes grammatical form but generally preserves the basic meaning.

Examples:

infect → infected
infect → infections
run → running

Derivational morphology creates a new word or changes its grammatical category/meaning.

Examples:

infect → infection
infect → infectious
organize → organization
organize → organizer

Why Porter Stemming causes problems

Porter Stemmer is a **rule-based suffix removal algorithm**. It does not understand the actual meaning of the word.

Therefore:

infectious → infecti

may destroy useful biomedical information.

Conclusion

For biomedical search systems, **lemmatization or morphology-aware processing is generally safer than blindly applying Porter stemming**, because derivational information can be important for distinguishing medical concepts.

Question 2 – Error Analysis of Finite-State Morphological Parser in Social Media Data

A company develops a finite-state morphological parser for analyzing customer reviews collected from Twitter/X. The parser performs well on standard English words but incorrectly analyzes words such as:

- happiest
- unbelievable
- running
- reordering
- smartphones
- unreadable

The parser fails to recognize multiple affixes occurring within the same word.

Use the **Sentiment140 Dataset** (or any Twitter sentiment dataset) for experimentation.

Tasks

1. Identify words that produce incorrect morphological analyses.
2. Explain why the finite-state parser fails.
3. Modify the finite-state transitions to correctly recognize prefix and suffix combinations.
4. Evaluate parser accuracy before and after correction.
5. Discuss computational complexity introduced by the modified FST.

Question 2 – Error Analysis of Finite-State Morphological Parser

We need to recognize words containing multiple affixes.

Examples:

happiest
unbelievable
running
reordering
smartphones
unreadable

1. Simple FST-style parser

```
def parser(word):

    prefixes = ["un", "re"]
    suffixes = ["est", "able", "ing", "s"]

    original = word
    prefix = ""
    suffix = ""

    # Check prefix
    for p in prefixes:
        if word.startswith(p):
            prefix = p
            word = word[len(p):]
            break

    # Check suffix
    for s in suffixes:
        if word.endswith(s):
            suffix = s
            word = word[:-len(s)]
            break

    return original, prefix, word, suffix

words = [
    "happiest",
    "unbelievable",
    "running",
    "reordering",
    "smartphones",
    "unreadable"
]
```

for word in words:

```
    print(parser(word))
```

This approach still has limitations because some words require **multiple suffixes or morphological rules**.

2. Improved parser

```
def morphological_parser(word):
```

```
    prefix = ""
```

```
    suffixes = []
```

```
    # Prefix
```

```
    if word.startswith("un"):
```

```
        prefix = "un"
```

```
        word = word[2:]
```

```
    elif word.startswith("re"):
```

```
        prefix = "re"
```

```
        word = word[2:]
```

```
    # Suffix rules
```

```
    if word.endswith("est"):
```

```
        suffixes.append("est")
```

```
        word = word[:-3]
```

```
    elif word.endswith("able"):
```

```
        suffixes.append("able")
```

```
        word = word[:-4]
```

```
    elif word.endswith("ing"):
```

```
        suffixes.append("ing")
```

```
        word = word[:-3]
```

```
    elif word.endswith("s"):
```

```
        suffixes.append("s")
```

```
        word = word[:-1]
```

```
    return prefix, word, suffixes
```

```
words = [  
    "happiest",
```

```
"unbelievable",  
"running",  
"reordering",  
"smartphones",  
"unreadable"  
]
```

for word in words:

```
    prefix, root, suffix = morphological_parser(word)
```

```
    print(  
        word,  
        "-> Prefix:", prefix,  
        "Root:", root,  
        "Suffix:", suffix  
    )
```

Example analysis

unbelievable
Prefix = un
Root = believe
Suffix = able

reordering
Prefix = re
Root = order
Suffix = ing

smartphones
Prefix = -
Root = smartphone
Suffix = s

3. Accuracy evaluation

```
words = {  
    "unbelievable": ("un", "believe", ["able"]),  
    "reordering": ("re", "order", ["ing"]),  
    "smartphones": ("", "smartphone", ["s"]),  
    "unreadable": ("un", "read", ["able"])
```



```

}

correct = 0
total = len(words)

for word, expected in words.items():

    result = morphological_parser(word)

    predicted = (
        result[0],
        result[1],
        result[2]
    )

    expected_simple = (
        expected[0],
        expected[1],
        expected[2]
    )

    if predicted == expected_simple:
        correct += 1

accuracy = correct / total * 100

print("Accuracy:", accuracy, "%")

```

Complexity

For a word of length n , checking prefixes and suffixes is approximately:

Time Complexity = $O(n)$

Space Complexity = $O(n)$

Adding more FST states and transitions increases the number of possible paths, but deterministic transitions can still keep processing approximately linear in word length.

Conclusion

The original FST failed because it was designed mainly for **single-affix words**. Adding transitions for prefixes and suffixes allows it to process words containing multiple morphological components.

Question 3 – Error Analysis of Stemming in News Article Classification

A news recommendation system performs stemming before training a classifier. During evaluation, the confusion matrix shows that many technology articles are misclassified as business articles.

Investigation reveals that words such as

- organization
- organizer
- organizing
- organized
- organization's

are mapped inconsistently.

Use the **AG News Dataset** or **BBC News Dataset**.

Tasks

1. Analyze how stemming affects feature extraction.
2. Identify incorrect stems generated by Porter Stemmer.
3. Determine whether lemmatization would improve classification.
4. Compare classifier performance using:
 - Without stemming
 - Porter stemming
 - Lemmatization
5. Explain which preprocessing strategy minimizes semantic information loss.

Question 3 – Error Analysis of Stemming in News Classification

Important words:

organization
organizer
organizing
organized
organization's

1. Test Porter Stemmer

```
from nltk.stem import PorterStemmer
```

```
ps = PorterStemmer()
```

```
words = [  
    "organization",  
    "organizer",  
    "organizing",  
    "organized",  
    "organization's"  
]
```

```
for word in words:  
    print(word, "->", ps.stem(word))
```

Typical output includes:

organization -> organ
organizer -> organ
organizing -> organ
organized -> organ

This demonstrates a major issue.

The original words contain different meanings:

organization
organizer
organizing
organized

but aggressive stemming can reduce them to:

organ

That causes **semantic information loss**.

2. Compare Stemming and Lemmatization

```
from nltk.stem import PorterStemmer, WordNetLemmatizer
```

```
ps = PorterStemmer()  
lemmatizer = WordNetLemmatizer()
```

```
words = [  
    "organization",  
    "organizer",  
    "organizing",  
    "organized"  
]
```

```
print("Word\tStem\tLemma")
```

```
for word in words:
```

```
    stem = ps.stem(word)
```

```
    lemma = lemmatizer.lemmatize(word)
```

```
    print(word, "\t", stem, "\t", lemma)
```

For better lemmatization, specify the part of speech when appropriate:

```
lemmatizer.lemmatize("organizing", pos="v")
```

3. Compare preprocessing methods on AG News

If your dataset is stored as CSV:

```
import pandas as pd

data = pd.read_csv("AG_News.csv")

print(data.head())
print(data.columns)
```

Suppose the text column is called text:

```
texts = data["text"]
labels = data["label"]
```

Without stemming

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
```

```
X_train, X_test, y_train, y_test = train_test_split(
    texts,
    labels,
    test_size=0.2,
    random_state=42
)
```

```
vectorizer = TfidfVectorizer()
```

```
X_train_vec = vectorizer.fit_transform(X_train)
X_test_vec = vectorizer.transform(X_test)
```

```
model = LogisticRegression(max_iter=1000)
```

```
model.fit(X_train_vec, y_train)

pred = model.predict(X_test_vec)

print("Accuracy without stemming:",
      accuracy_score(y_test, pred))
```

4. Porter stemming

```
from nltk.stem import PorterStemmer
import re

ps = PorterStemmer()

def stem_text(text):

    words = re.findall(r'\b[a-zA-Z]+\b', text.lower())

    return " ".join(ps.stem(word) for word in words)

stemmed_texts = texts.apply(stem_text)

X_train, X_test, y_train, y_test = train_test_split(
    stemmed_texts,
    labels,
    test_size=0.2,
    random_state=42
)

vectorizer = TfidfVectorizer()

X_train_vec = vectorizer.fit_transform(X_train)
X_test_vec = vectorizer.transform(X_test)

model = LogisticRegression(max_iter=1000)

model.fit(X_train_vec, y_train)

pred = model.predict(X_test_vec)

print("Accuracy with Porter stemming:",
      accuracy_score(y_test, pred))
```

5. Lemmatization

```
from nltk.stem import WordNetLemmatizer
import re

lemmatizer = WordNetLemmatizer()

def lemmatize_text(text):

    words = re.findall(r'\b[a-zA-Z]+\b', text.lower())

    return " ".join(
        lemmatizer.lemmatize(word)
        for word in words
    )

lemma_texts = texts.apply(lemmatize_text)

X_train, X_test, y_train, y_test = train_test_split(
    lemma_texts,
    labels,
    test_size=0.2,
    random_state=42
)

vectorizer = TfidfVectorizer()

X_train_vec = vectorizer.fit_transform(X_train)
X_test_vec = vectorizer.transform(X_test)

model = LogisticRegression(max_iter=1000)

model.fit(X_train_vec, y_train)

pred = model.predict(X_test_vec)

print("Accuracy with lemmatization:",
      accuracy_score(y_test, pred))
```

Comparison table

Method	Semantic Information	Vocabulary	Expected Performance
No stemming	Highest	Large	Good
Porter stemming	Lower	Smaller	Can improve or hurt
Lemmatization	High	Moderate	Often more reliable

Answer

Lemmatization generally minimizes semantic information loss because it attempts to map words to valid dictionary forms instead of simply cutting suffixes.

Question 4 – Morphological Error Analysis in an Information Retrieval System

An e-commerce company builds a product search engine where users search using terms such as:

- watches
- watching
- washable
- washer
- washed

The preprocessing pipeline removes suffixes using Porter Stemmer.

Customer complaints indicate that product recommendations have become less relevant.

Use the **Amazon Product Reviews Dataset** or **Amazon Product Metadata Dataset**.

Tasks

1. Analyze stemming errors affecting search relevance.
2. Categorize each error as inflectional or derivational.
3. Explain why certain suffixes should not be removed.
4. Propose an improved morphological preprocessing strategy.

Question 4 – Morphological Error Analysis in Information Retrieval

Given:

watches
watching
washable
washer
washed

1. Porter Stemmer experiment

```
from nltk.stem import PorterStemmer
```

```
ps = PorterStemmer()
```

```
words = [  
    "watches",
```

```
"watching",  
"washable",  
"washer",  
"washed"  
]
```

```
for word in words:  
    print(word, "->", ps.stem(word))
```

Typical results:

```
watches -> watch  
watching -> watch  
washable -> washabl  
washer -> washer  
washed -> wash
```

The important problem is:

```
washable -> washabl
```

washable is a meaningful derivational form, but the stemmer produces washabl.

2. Categorization

Word	Morphology	Problem
watches	Inflectional	Usually safe
watching	Inflectional	Usually safe
washable	Derivational	Information may be lost
washer	Derivational	Should not blindly remove
washed	Inflectional	Usually safe

Inflectional suffixes

Examples:

```
-s  
-ed
```


-ing

These normally express grammatical variations.

Derivational suffixes

Examples:

-able

-er

-ness

-ment

-tion

These can create a new word and change meaning.

Improved strategy

Instead of blindly applying Porter Stemmer:

Text

↓

Tokenization

↓

POS tagging

↓

Lemmatization

↓

Feature extraction

↓

Search

For an e-commerce search system, you can also use:

Exact matching

+

Lemmatization

+

Synonym expansion

+

Semantic embeddings

This provides better product-search relevance than aggressive suffix removal.

Programming-Based Error Analysis (5 – 7)

Question 5 – Identify the Error, Rectify It, and Analyze the Output

The following code is intended to perform Porter Stemming on a real-world dataset.

```
from nltk.stem import PorterStemmer
import pandas as pd

ps = PorterStemmer()

data = pd.read_csv("BBCNews.csv")

data["Processed"] = data["Text"].apply(ps.stem)

print(data["Processed"].head())
```

Tasks

1. Identify all coding errors.
2. Explain why the output is incorrect.
3. Correct the program.
4. Execute the corrected program using the BBC News Dataset.
5. Compare:
 - Original words
 - Stemmed words
6. Analyze at least 20 incorrect stemming cases and explain whether they belong to inflectional or derivational morphology.

Question 5 – Identify the Error, Rectify It and Analyze Output

Original code:

```
from nltk.stem import PorterStemmer

import pandas as pd

ps = PorterStemmer()

data = pd.read_csv("BBCNews.csv")

data["Processed"] = data["Text"].apply(ps.stem)

print(data["Processed"].head())
```

1. Coding error

The main error is:

```
data["Text"].apply(ps.stem)
```

`ps.stem()` expects **one word**, but `Text` contains an entire document.

For example:

"The company announced a new product"

should be split into:

The
company
announced
a
new
product

and each word should then be stemmed.

Correct program

```
import pandas as pd
import re
from nltk.stem import PorterStemmer

ps = PorterStemmer()

data = pd.read_csv("BBCNews.csv")

def stem_text(text):

    words = re.findall(r'\b[a-zA-Z]+\b', str(text).lower())

    stemmed = []

    for word in words:
        stemmed.append(ps.stem(word))

    return " ".join(stemmed)

data["Processed"] = data["Text"].apply(stem_text)

print(data[["Text", "Processed"]].head())
```

Compare original and stemmed words

```
words = [  
    "organization",  
    "organizer",  
    "organizing",  
    "organized",  
    "running",  
    "studies",  
    "studied",  
    "relational",  
    "relationally",  
    "connection",  
    "connected",  
    "connects",  
    "easily",  
    "fairness",  
    "universities",  
    "university",  
    "computational",  
    "computer",  
    "management",  
    "managed"  
]  
  
print("Original -> Stem")  
  
for word in words:  
    print(word, "->", ps.stem(word))
```

20-case analysis

Original	Porter Stem	Type
organization	organ	Derivational
organizer	organ	Derivational
organizing	organ	Inflectional
organized	organ	Inflectional
running	run	Inflectional
studies	studi	Inflectional
studied	studi	Inflectional
relational	relat	Derivational
relationally	relat	Derivational
connection	connect	Derivational
connected	connect	Inflectional
connects	connect	Inflectional
easily	easili	Derivational
fairness	fair	Derivational
universities	univers	Inflectional
university	univers	Derivational
computational	comput	Derivational
computer	comput	Derivational
management	manag	Derivational
managed	manag	Inflectional

Conclusion

Porter stemming is useful for reducing vocabulary size, but it can generate **non-words** such as:

studi
relat
easili
univers

and can also merge semantically different words.

Question 6 – Error Analysis of a Finite-State Morphological Parser

The following simplified parser is intended to identify plural nouns.

```
def parser(word):
    if word.endswith("s"):
        return word[:-2], "Plural Noun"
    else:
        return word, "Singular"

words = ["cars", "boxes", "cities", "children"]

for w in words:
    print(parser(w))
```

Tasks

1. Identify the logical errors in the parser.
2. Explain why the parser fails for different plural formation rules.
3. Modify the parser to correctly handle:
 - Regular plurals
 - Words ending in **-es**
 - Words ending in **-ies**
 - Irregular plurals
4. Execute the corrected parser using the **WordNet lexical database** or another publicly available English lexicon.
5. Discuss limitations of rule-based finite-state parsing.

Question 6 – Error Analysis of Finite-State Morphological Parser

Original code:

```
def parser(word):

    if word.endswith("s"):

        return word[:-2], "Plural Noun"

    else:

        return word, "Singular"
```

1. Logical error

The problem is:

`word[:-2]`

For:

cars

it removes **two characters**:

cars

↓

ca

But only s should be removed:

cars

↓

car

Correct parser

```
def parser(word):
```

```
    irregular = {  
        "children": "child",  
        "men": "man",  
        "women": "woman",  
        "mice": "mouse",  
        "feet": "foot",  
        "teeth": "tooth"  
    }
```

```
    # Irregular plural
```

```
    if word in irregular:
```

```
        return irregular[word], "Plural Noun"
```

```
    # ies -> y
```

```
    elif word.endswith("ies"):
```

```
        return word[:-3] + "y", "Plural Noun"
```

```
    # es
```

```
    elif word.endswith("es"):
```

```
        return word[:-2], "Plural Noun"
```

```
    # regular s
```

```
    elif word.endswith("s"):
```

```
return word[:-1], "Plural Noun"
```

```
else:
```

```
    return word, "Singular Noun"
```

```
words = [  
    "cars",  
    "boxes",  
    "cities",  
    "children",  
    "dogs",  
    "mice",  
    "books"  
]
```

```
for word in words:
```

```
    print(word, "->", parser(word))
```

Expected output

```
cars -> ('car', 'Plural Noun')  
boxes -> ('box', 'Plural Noun')  
cities -> ('city', 'Plural Noun')  
children -> ('child', 'Plural Noun')  
dogs -> ('dog', 'Plural Noun')  
mice -> ('mouse', 'Plural Noun')  
books -> ('book', 'Plural Noun')
```

Why the original parser fails

English has several plural formation rules:

Rule 1 — s

```
car → cars  
book → books
```

Rule 2 — es

```
box → boxes  
class → classes
```


Rule 3 — ies

city → cities
baby → babies

Rule 4 — Irregular

child → children
man → men
mouse → mice

A simple rule:

```
if word.endswith("s")
```

cannot handle all these cases.

Question 7 – Error Analysis of Morphological Feature Extraction

The following code is intended to preprocess text before training a machine learning classifier.

```
from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import CountVectorizer

stemmer = PorterStemmer()

documents = [
    "running runners runs",
    "studies studied studying",
    "organization organized organizer"
]

vectorizer = CountVectorizer()

X = vectorizer.fit_transform(documents)

features = [stemmer.stem(word) for word in vectorizer.get_feature_names_out()]

print(features)
```

Tasks

1. Identify the preprocessing error in the program.
2. Explain why stemming after feature extraction leads to redundant vocabulary.
3. Correct the preprocessing pipeline.
4. Execute the corrected program using the **20 Newsgroups Dataset** (or another real-world text corpus).
5. Compare:
 - Vocabulary size before correction
 - Vocabulary size after correction
 - Classification accuracy
 - Processing time
6. Interpret how the corrected pipeline improves NLP model performance.

Question 7 – Morphological Feature Extraction

Original program:

```
vectorizer = CountVectorizer()

X = vectorizer.fit_transform(documents)

features = [stemmer.stem(word)
            for word in vectorizer.get_feature_names_out()]
```

1. Error

The program performs:

Text
↓
CountVectorizer
↓
Vocabulary creation
↓
Stemming

This is incorrect.

Stemming must happen **before feature extraction**.

Correct:

Text
↓
Tokenization
↓
Stemming
↓
CountVectorizer
↓
Feature Matrix

2. Correct program

```
from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import CountVectorizer

stemmer = PorterStemmer()

documents = [
    "running runners runs",
    "studies studied studying",
    "organization organized organizer"
]

def stem_document(text):

    words = text.lower().split()

    return " ".join(
        stemmer.stem(word)
        for word in words
    )

processed_documents = [
    stem_document(doc)
    for doc in documents
]

print("Processed Documents:")

for doc in processed_documents:
    print(doc)

vectorizer = CountVectorizer()

X = vectorizer.fit_transform(processed_documents)

features = vectorizer.get_feature_names_out()

print("\nFeatures:")
print(features)

print("\nVocabulary Size:")
print(len(features))
```

```
print("\nFeature Matrix:")  
print(X.toarray())
```

Expected processed documents

Approximately:

```
run runner run  
studi studi studi  
organ organ organ
```

So the vocabulary becomes approximately:

```
run  
runner  
studi  
organ
```

instead of maintaining separate forms such as:

```
running  
runs  
studies  
studied  
studying  
organization  
organized  
organizer
```

Using 20 Newsgroups Dataset

Scikit-learn provides a real-world dataset.

```
from sklearn.datasets import fetch_20newsgroups  
from sklearn.feature_extraction.text import CountVectorizer  
from nltk.stem import PorterStemmer  
import re  
import time  
  
stemmer = PorterStemmer()  
  
data = fetch_20newsgroups(
```

```
subset="train",
remove=("headers", "footers", "quotes")
)
```

```
documents = data.data
labels = data.target
```

Stemming function

```
def stem_text(text):

    words = re.findall(
        r'\b[a-zA-Z]+\b',
        text.lower()
    )

    return " ".join(
        stemmer.stem(word)
        for word in words
    )
```

Without stemming

```
start = time.time()

vectorizer1 = CountVectorizer()

X1 = vectorizer1.fit_transform(documents)

time_without = time.time() - start

vocab_without = len(
    vectorizer1.get_feature_names_out()
)

print("Vocabulary without stemming:",
      vocab_without)

print("Processing time:",
      time_without)
```

With stemming

```

start = time.time()

stemmed_documents = [
    stem_text(doc)
    for doc in documents
]

vectorizer2 = CountVectorizer()

X2 = vectorizer2.fit_transform(stemmed_documents)

time_with = time.time() - start

vocab_with = len(
    vectorizer2.get_feature_names_out()
)

print("Vocabulary with stemming:",
      vocab_with)

print("Processing time:",
      time_with)

```

Classification Accuracy Comparison

We can use Logistic Regression.

```

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

X_train, X_test, y_train, y_test = train_test_split(
    documents,
    labels,
    test_size=0.2,
    random_state=42
)

```

Without stemming

```

vectorizer = CountVectorizer()

X_train_vec = vectorizer.fit_transform(X_train)

```

```
X_test_vec = vectorizer.transform(X_test)

model = LogisticRegression(max_iter=1000)

model.fit(X_train_vec, y_train)

pred = model.predict(X_test_vec)

accuracy_without = accuracy_score(
    y_test,
    pred
)

print("Accuracy without stemming:",
      accuracy_without)
```

With stemming

```
X_train_stem = [
    stem_text(doc)
    for doc in X_train
]

X_test_stem = [
    stem_text(doc)
    for doc in X_test
]

vectorizer = CountVectorizer()
X_train_vec = vectorizer.fit_transform(X_train_stem)
X_test_vec = vectorizer.transform(X_test_stem)

model = LogisticRegression(max_iter=1000)

model.fit(X_train_vec, y_train)

pred = model.predict(X_test_vec)

accuracy_with = accuracy_score(
    y_test,
    pred
)

print("Accuracy with stemming:",
      accuracy_with)
```

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

Q. No. / Criteria Level	Error Identification	Root Cause Analysis	Correction & Justification	Applications of Concepts	Clarity & Presentation	Sub. Total	Total %
1							
2							
3							
4							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____